

SSR and Web Components— A Contentious Couple



Vishal Sharma · [Follow](#)

5 min read · Jul 31, 2024



Listen



Share

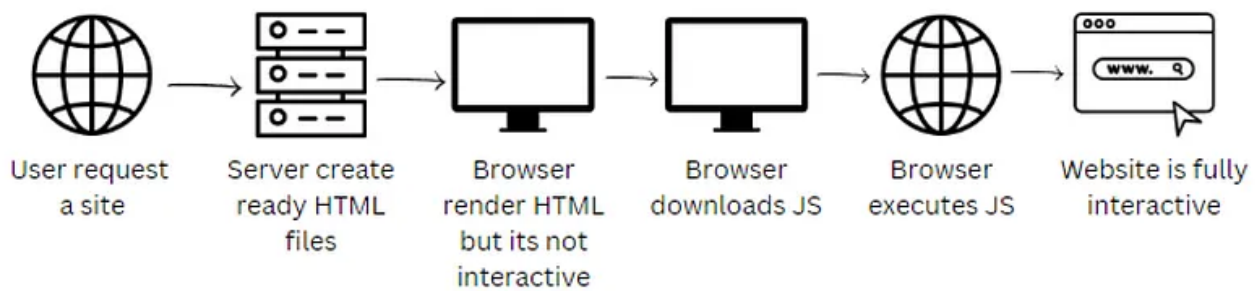
Usually, we talk about the technologies that work best together. This article is the exact opposite where I would like to discuss the problems we faced using web components in a server-side rendered application and some possible resolution of those.



First, a quick overview of both the technologies

Server Side Rendering

Server-side rendering allows generating the whole HTML on the server and then hydrating on the client for faster page loads with fewer blocking scripts.



Server-side Rendering provides a lot of benefits over traditional Client-side Rendered apps.

- **Performance** — No big chunk of JS is blocking the browser's HTML rendering, meaning the user can see the app content immediately, improving the loading time and thus the perceived performance.
- **Better SEO Ranking** — Unlike CSR apps, the server returns the whole HTML with all the nested links, thus getting better SEO ranking.
- **Security** — All the template logic is on the server side which means you don't need to expose the secrets or tokens on the client side leading to improved data security.

Web Components

Web components are a set of web platform APIs that allow you to create new custom, reusable, encapsulated HTML tags to use in web pages and web apps.

```

▼ <my-drawer is="closed" class="hydrated">
  ▼ #shadow-root (open)
    ▼ <div class="drawer">
      ▶ <div class="drawer-header-container"> ... </div>
      ▶ <div class="drawer-content-container"> ... </div>
      ▶ <div class="button-container"> ... </div>
    </div>
    <div class="backdrop"></div>
    <div slot="drawer-header">I am the Drawer Header</div> slot
    <div slot="drawer-content">I am the Drawer Content</div> slot
  </my-drawer>
  
```

Shadow root is a set of JavaScript APIs for attaching an encapsulated “shadow” DOM tree to an element that is rendered separately from the main document DOM. In

this way, you can keep an element's features private, so they can be scripted and styled without the fear of collision with other parts of the document.

To make browsers understand web components, they need to get registered on the client:

```
customElements.define("my-drawer", MyDrawer);
```

Benefits of using web components:

- **Framework Agnostic** — Can work with any client-side frameworks. We just need to register the web component on the client.
- **Style Encapsulation** — With shadow root encapsulation, you don't need to fear style ambiguity with the web component.
- **Performance** — There is no need for a JS framework to be served on the wire. We don't need any hydration to work as well which also makes it more compatible with all the browsers by default.

Now we have understood both the technologies to some extent and their

[Open in app](#) ↗

[Sign up](#)

[Sign in](#)

Medium

 Search



world experience of using web components using stencil JS in a Design system library which was included in a Next JS SSR app. This is not against any technology or framework.

Let us move to the issues now:

- Search Engine Optimisation

SEO is one of the main advantages of using SSR. On publishing the content, the crawlers crawl the page, index it and if the page meets the ranking criteria, it will be shown in the search results.

Bots only find deep URLs in the site by scanning the document object model (DOM) for links. The bot doesn't click on anything, so you need to use links in your

rendered HTML to tell the bot about all the pages.

Mainly Google and Bing are advanced enough to index JS websites aka CSR websites. If you are working on a global app, you don't want to rely on only Google even if it has the maximum share.

Now, when we create a web component for a link and serve it on the wire

```
<my-link to="page1.com" title="go to page 1"></my-link>
```

the crawlers will not understand that this is a link available on your website and may not be indexed which will lead to less SEO ranking.

Resolution — Slot based components

```
<my-link>  
  <a slot="anchor" href="page1.com">go to page 1</a>  
</my-link>
```

The slot-based approach uses native HTML markup and the links are now included inside the web components as a slot, allowing crawlers to fully understand page structure. With this approach, you continue to have the style encapsulation with SEO as well but then any change in slot does not update the web component functionality which adds to the complexity of the solution.

- Performance Degradation

Server-side rendered apps and web components both individually have performance as their USP. But client-side registration of web components causes delays in LCP making even server-side rendered apps slower in performance. The registration process for a big list of web components can become render-blocking for the rest of the page, displaying a blank page for a significant period.

Resolution — Lazy registration

- we can register only the components that we want first to be visible on the page instead of registering all e.g. in components in the header etc.

- But this adds to the complexity to work with which can cause bad developer experience and wasted time.
- We can even delay the registration for all the components but that leads to our next issue Flash of Unstyled Components.

- Flash of Unstyled Content

A flash of unstyled content (FOUC, or flash of unstyled text) is an instance where a web page appears briefly with the browser's default styles before loading an external CSS stylesheet. The slot-based approach to improve SEO causes FoUC as encapsulated styles do not get applied till the time component is registered on the client.

Resolution

- Hide the slot element using the CSS visibility rule
- In the case of nested elements, none of the elements should have `visible` CSS property

- Hydration Errors

In an SSR app, If the server and client markups do not match on the initial render, a hydration error is thrown. There can be multiple reasons for hydration error as mentioned by [this](#) Next JS page.

But while building the design system using web components, we realised that the common reasons for the errors were:

- Use of browser global methods that are not present in the server-side node js app. If the design components are being built by a different team, this error is bound to happen.
- Attribute properties mismatch because of client-level registration of web components. e.g. if you are adding a `dir` attribute to the host element of a web component, a change in the value of `dir` on the client causes the hydration error.

Resolution

- Don't use or check for the presence of a window object `typeof window !== 'undefined'` in the web component rendering logic.
- Avoid using dynamic attributes on web components' host element.

A few other issues apart from the major one discussed above were related to **Cumulative Layout Shift**, **Library version mismatch in Micro frontend architecture**, **Unit testing the react components using web components** and **Accessibility**.

Given, all the issues, I would like to end this post by adding some recommendations.

Recommendations

- Don't try to fit in a solution. If ur whole ecosystem is in one framework, why even invest in a different one just for the sake of reusability.
- Evaluate the solution with different scenarios before jumping to the conclusion.
- Involve stakeholders from different teams to understand needs.

Ssr

Web Components

Web Performance

SEO

Hydration Errors



Follow

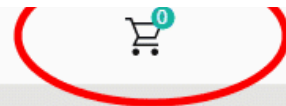


Written by Vishal Sharma

129 Followers

Front End Engineer at /thoughtworks.

More from Vishal Sharma



Tennis Shoes

129.95 **99.95**

ADD TO CART



Running Shoes

129.95 **99.95**

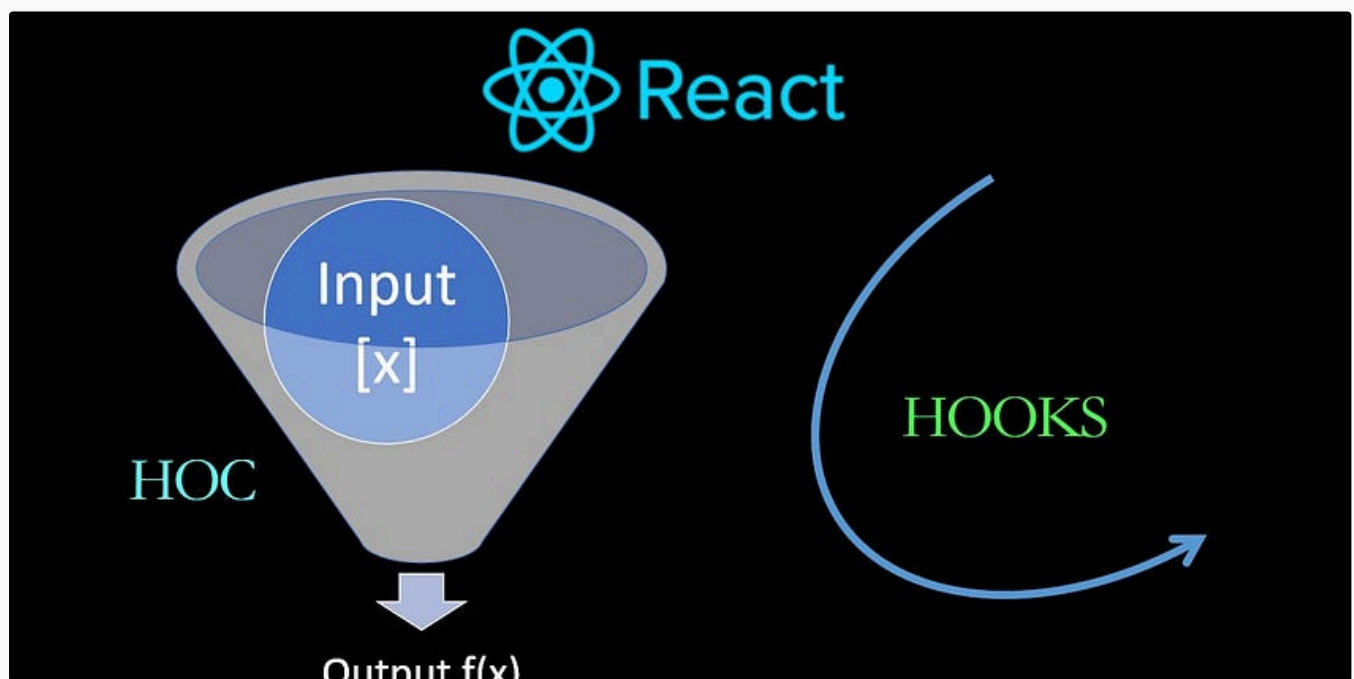
ADD TO CART

 Vishal Sharma

5 Different Techniques for Cross Micro Frontend Communication

The title itself signifies what we are going to discuss in this article. But before we even start, let's try to understand why cross-micro...

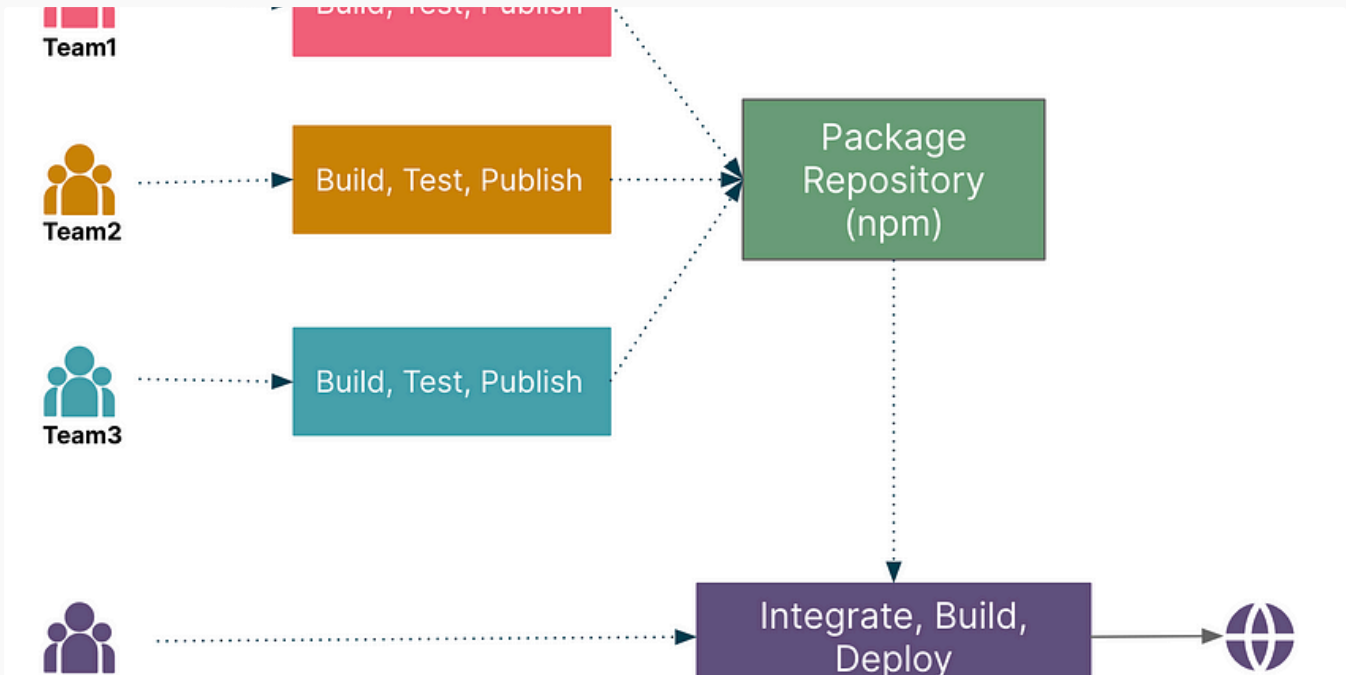
Jun 12, 2022  247  4



 Vishal Sharma

Following DRY principle for Api Calls in React

In most of the React projects, we have components where we need to fetch the data from an api on mount. React provides us lifecycle...



Vishal Sharma

How to Compose Micro Frontends at Build Time

We have been building npm packages and using those inside the frontend application for eternity now. Build time composition of Micro...



```
package.json

{
  "name": "ts-eslint-jest-setup",
  "version": "1.0.0",
  "main": "index.js",
  "license": "MIT"
}
```

Vishal Sharma

Setting up Typescript + ESLint + Jest Project

A lot of node projects have a setup which usually include Typescript for static type checking, esLint for static code analysis and Jest as...

Jul 7, 2021 🖱 51 💬 2



See all from Vishal Sharma

Recommended from Medium



Arnold Gunter

These 7 Small JavaScript Bugs Almost Broke the Internet

On March 22, 2016, in California USA, a JavaScript developer named Azer Koçulu removed the small npm package “left-pad”, and caused chaos...

★ Oct 5 🖱 187 💬 2





Hima Chitalia in JavaScript in Plain English

Understanding Array.of: A Deep Dive into Creating Arrays in JavaScript

In JavaScript, creating arrays has always been a fundamental part of working with data. The traditional Array constructor allows us to...



6d ago



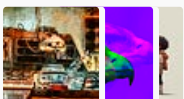
60



1



Lists



What is ChatGPT?

9 stories · 447 saves



Generative AI Recommended Reading

52 stories · 1432 saves



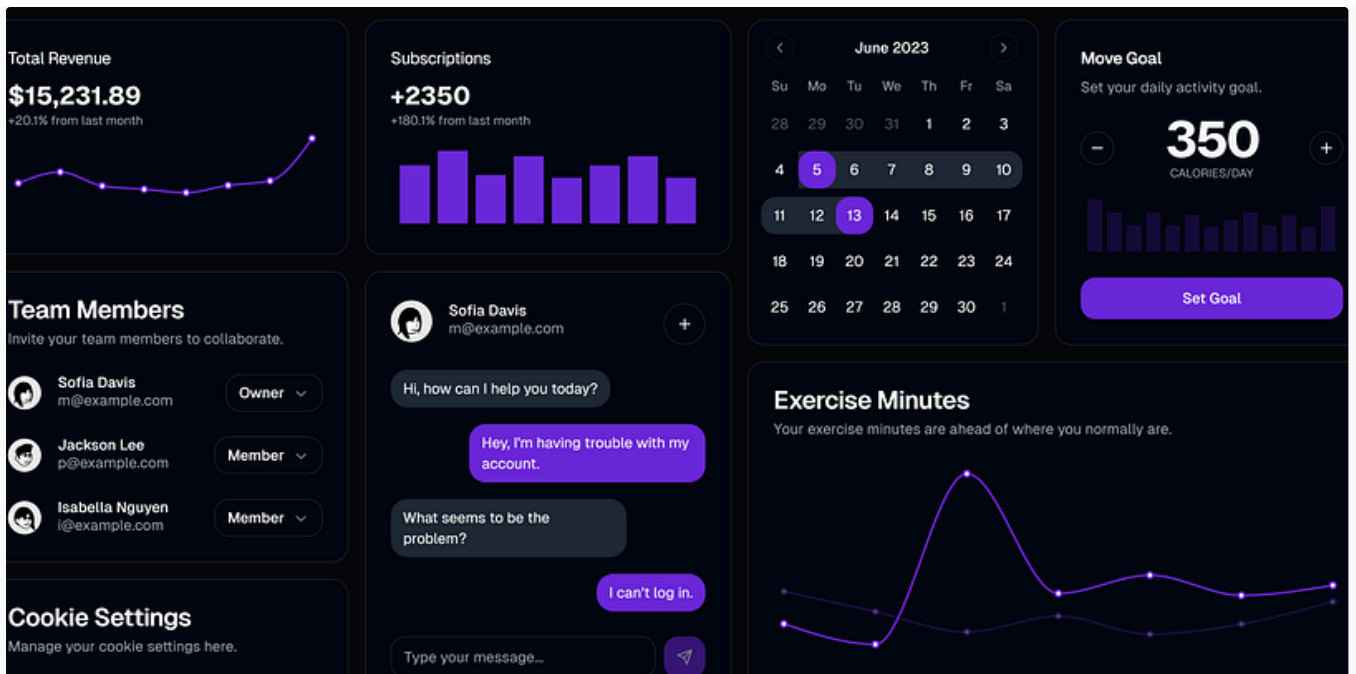
MODERN MARKETING

193 stories · 884 saves



Staff Picks

750 stories · 1372 saves

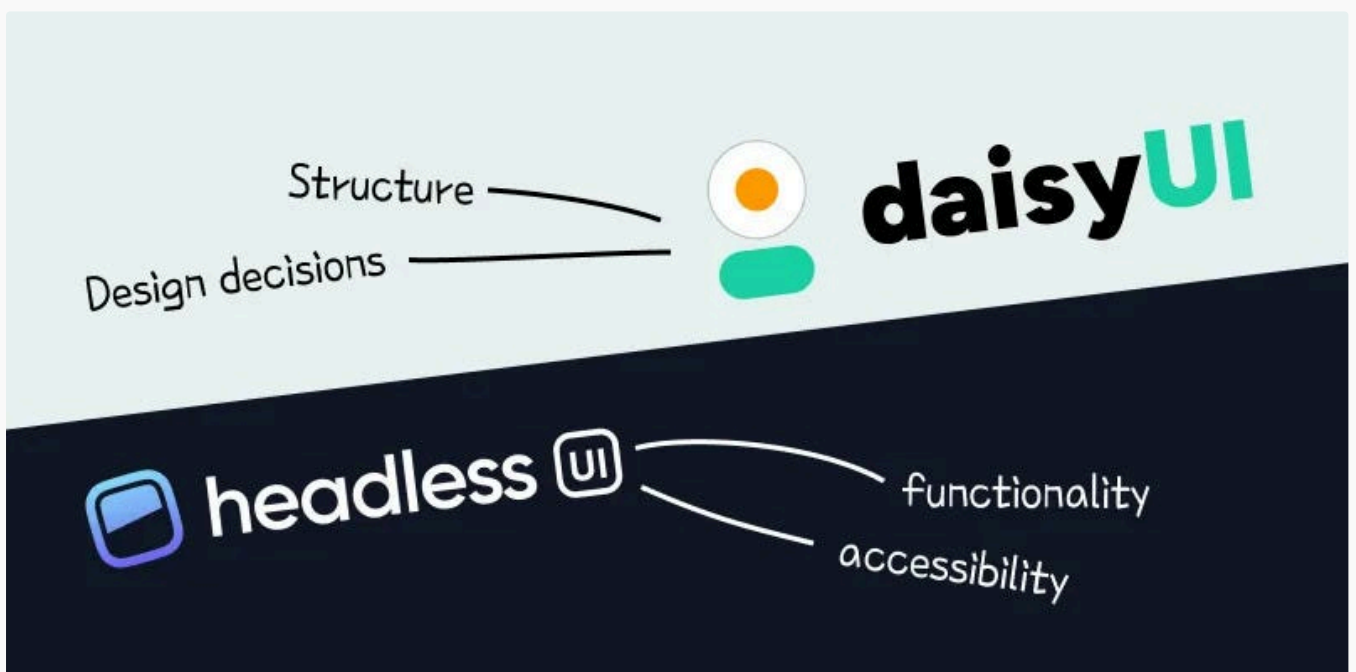


 Tari Ibaba in Coding Beauty

This New UI Framework Is An Absolute Game Changer 🤖

The new shadcn/ui framework completely transforms how we design and build modern web apps.

🌟 Oct 5 🖱️ 1K 💬 13



 Asian Digital Hub in DevSphere

Top 5 UI Component Libraries for Tailwind and Next.js

Source: DaisyUi

Amazon.com

Seattle, WA

Software Development Engineer

Mar. 2020 – May 2021

- Developed Amazon checkout and payment services to handle traffic of 10 Million daily global transactions
- Integrated Iframes for credit cards and bank accounts to secure 80% of all consumer traffic and prevent CSRF, cross-site scripting, and cookie-jacking
- Led Your Transactions implementation for JavaScript front-end framework to showcase consumer transactions and reduce call center costs by \$25 Million
- Recovered Saudi Arabia checkout failure impacting 4000+ customers due to incorrect GET form redirection

Projects

NinjaPrep.io (React)

- Platform to offer coding problem practice with built in code editor and written + video solutions in React
- Utilized Nginx to reverse proxy IP address on Digital Ocean hosts
- Developed using Styled-Components for 95% CSS styling to ensure proper CSS scoping
- Implemented Docker with Seccomp to safely run user submitted code with < 2.2s runtime

HeatMap (JavaScript)

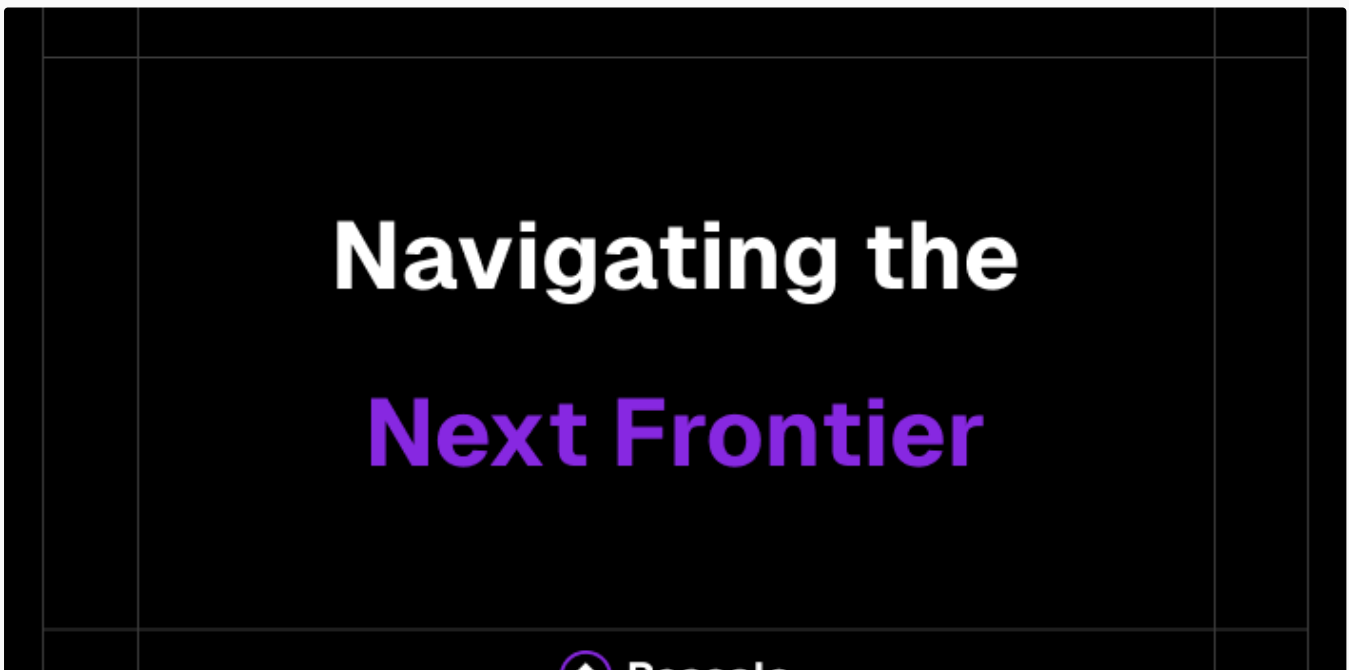
- Visualized Google Takeout location data of location history using Google Maps API and Google Maps heatmap code with React
- Included local file system storage to reliably handle 5mb of location history data
- Implemented Express to include routing between pages and jQuery to parse Google Map and implement heatmap overlay



Alexander Nguyen in Level Up Coding

The resume that got a software engineer a \$300,000 job at Google.

1-page. Well-formatted.



Z4NR34L - Mateusz Janota in Rescale

Navigating the Next Frontier: Road to Next.js 15 and Embracing Asynchronous Revolution

The release of Next.js 15 (Canary — introduced in v15.0.0-canary.171) marks a pivotal moment for developers.

Oct 5  13



See more recommendations